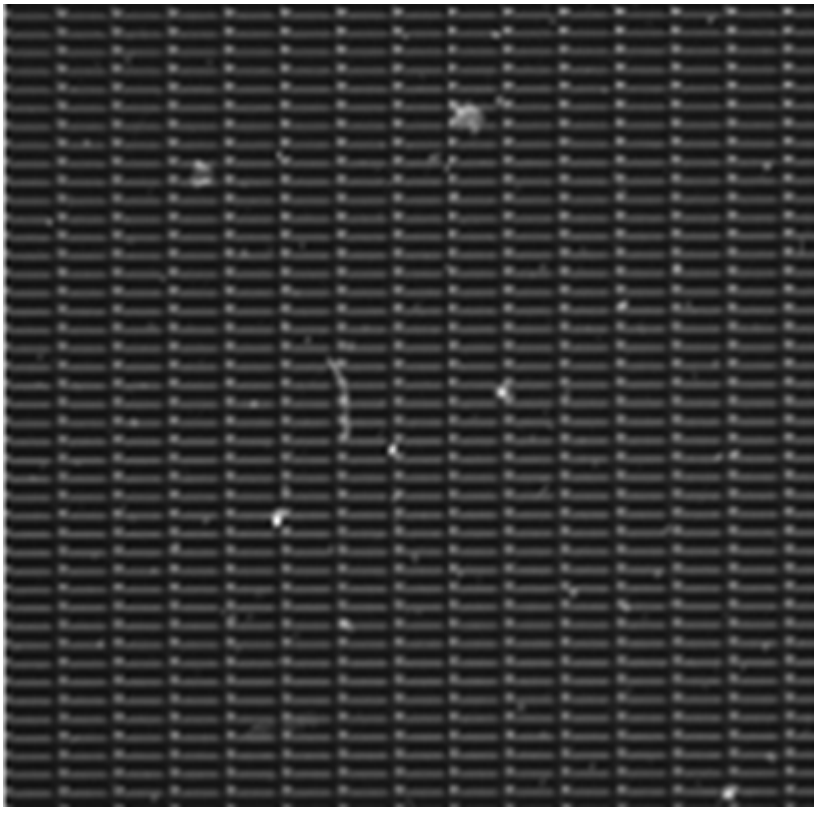
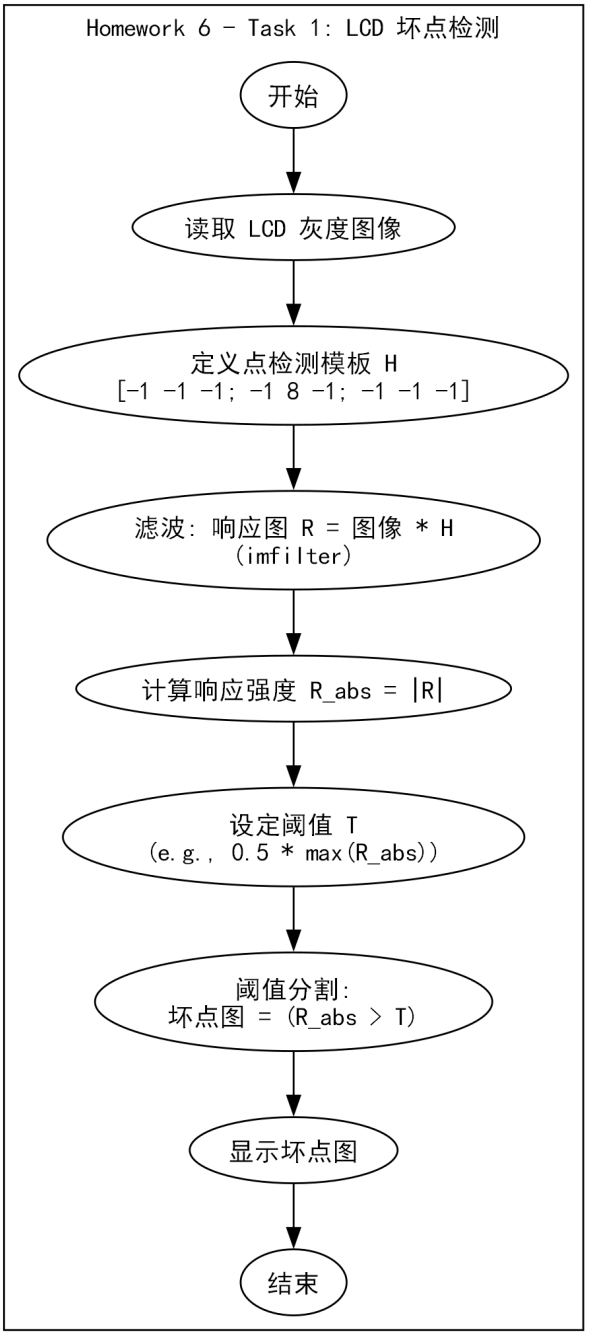
数字图像处理 作业6

姓名：卞常灿 班级：自动化3班 学号：2023312216

1. 设计LCD屏幕坏点检测算法，给出流程图，以及在matlab或者opencv中实现代码；



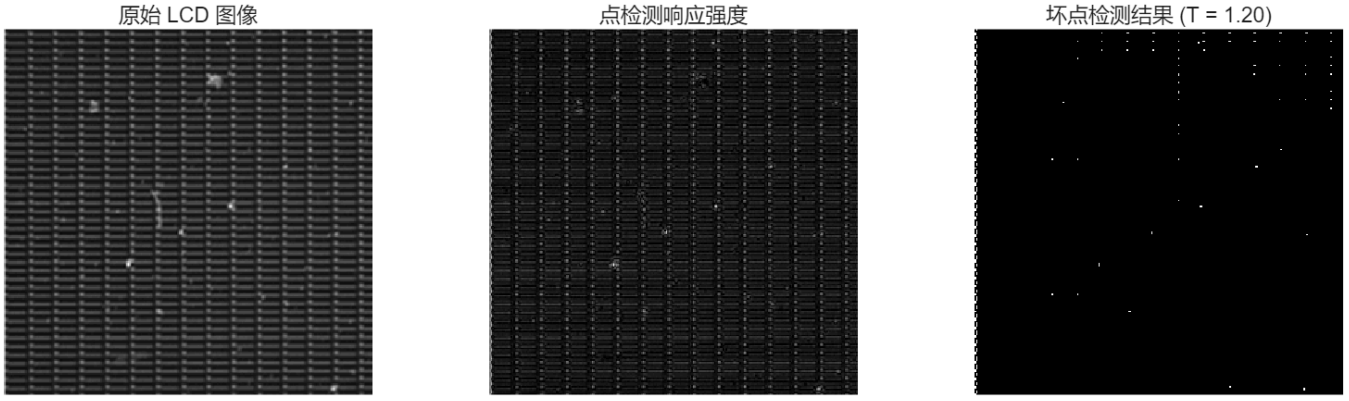



```
clear; clc; close all;

% 1. 读取图像
img_lcd = imread('lcd_bad_pixels.bmp');
if size(img_lcd, 3) == 3
    gray_lcd = rgb2gray(img_lcd);
else
    gray_lcd = img_lcd;
end
figure;
subplot(1,3,1); imshow(gray_lcd); title('原始 LCD 图像');

% 2. 点检测滤波
% 点检测模板 (Laplacian 类型)
h_point = [-1 -1 -1; -1 8 -1; -1 -1 -1];
```

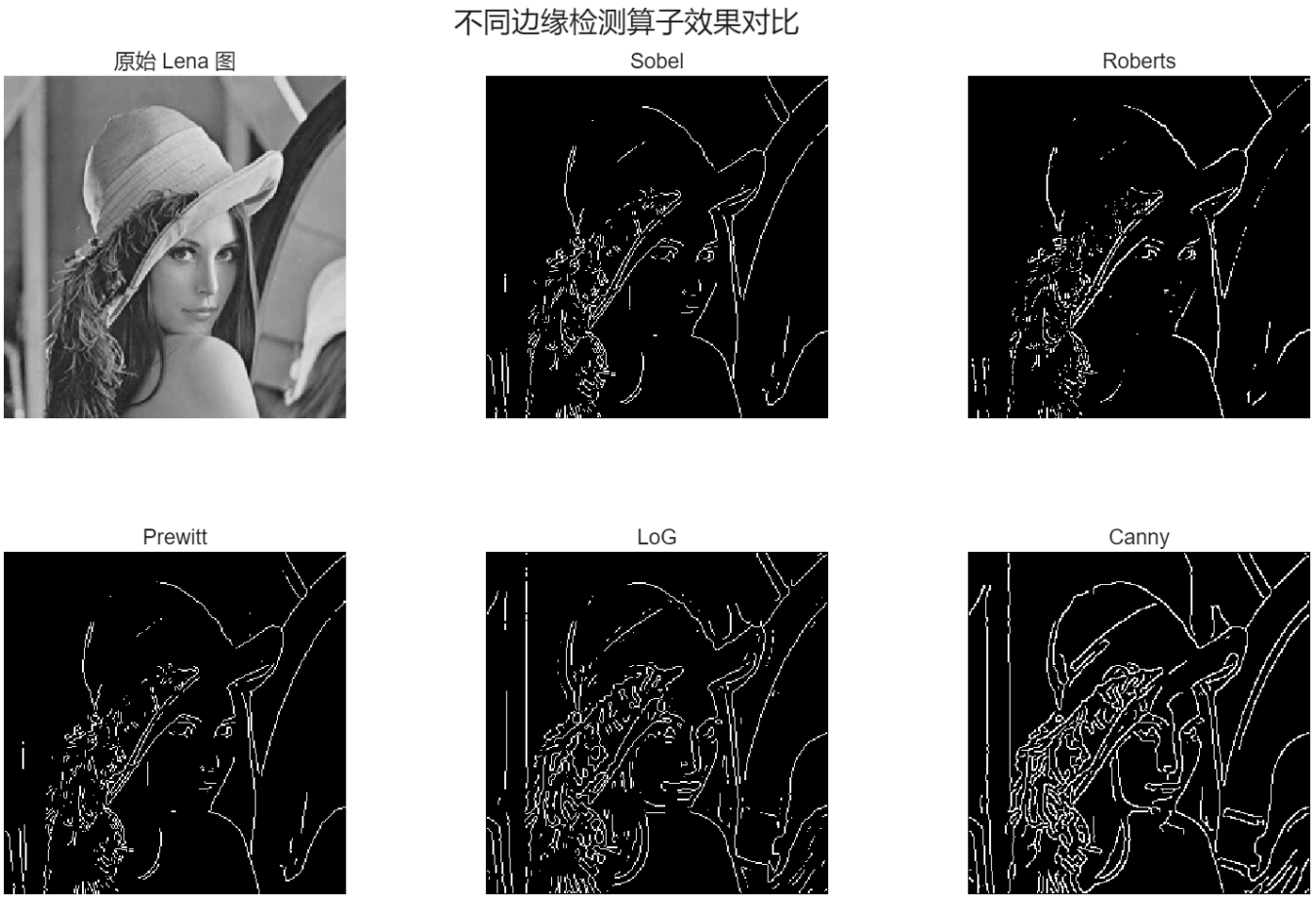
```
16
17  % 使用 imfilter 进行卷积 (replicate 边界处理)
18  % 将图像转为 double 类型以进行滤波计算
19  response = imfilter(im2double(gray_lcd), h_point, 'replicate');
20
21  % 3. 计算响应强度
22  abs_response = abs(response);
23  subplot(1,3,2); imshow(abs_response, []); title('点检测响应强度'); % 使用 [] 自动调整显示范围
24
25  % 4. 阈值分割
26  % 阈值 T 可以根据响应强度的直方图或经验调整
27  % 例如，取最大响应值的 50% 作为阈值
28  max_resp = max(abs_response(:));
29  threshold_T = 0.5 * max_resp; % 0.5 是一个可调参数
30
31  bad_pixels_map = abs_response > threshold_T;
32
33  subplot(1,3,3); imshow(bad_pixels_map); title(['坏点检测结果 (T = ' num2str(threshold_T,
    '%.2f') ')']);
```



2. matlab or opencv 对Lena的图片进行边缘提取，使用Sobel, Roberts, Prewitt, LoG, Canny；

```
1   clear; clc; close all;
2
3   % 1. 读取 Lena 图像
4   img_lena = imread('lena.bmp');
5   if size(img_lena, 3) == 3
6       gray_lena = rgb2gray(img_lena);
7   else
8       gray_lena = img_lena;
9   end
10
11  % 2. 应用不同的边缘检测算子
12  edge_sobel = edge(gray_lena, 'sobel');
13  edge_roberts = edge(gray_lena, 'roberts');
14  edge_prewitt = edge(gray_lena, 'prewitt');
15  edge_log = edge(gray_lena, 'log'); % Laplacian of Gaussian
16  edge_canny = edge(gray_lena, 'canny');
17
18  % 3. 显示结果
19  figure;
```

```
subplot(2,3,1); imshow(gray_lena); title('原始 Lena 图');
subplot(2,3,2); imshow(edge_sobel); title('Sobel');
subplot(2,3,3); imshow(edge_roberts); title('Roberts');
subplot(2,3,4); imshow(edge_prewitt); title('Prewitt');
subplot(2,3,5); imshow(edge_log); title('LoG');
subplot(2,3,6); imshow(edge_canny); title('Canny');
sgtitle('不同边缘检测算子效果对比');
```



3. 改进第三次课程实验中的圆检测方法，给出代码实现

```
clear; clc; close all;

% 1. 读取包含圆的图像
img_circle = imread('circle.png');
if size(img_circle, 3) == 3
    gray_circle = rgb2gray(img_circle);
else
    gray_circle = img_circle;
end

figure;
imshow(gray_circle); title('原始圆图像');

% 2. 使用 imfindcircles 进行圆检测
% --- 参数调整 ---
% (需要根据实际图像估计半径范围)
image_height = size(gray_circle, 1);
image_width = size(gray_circle, 2);
```

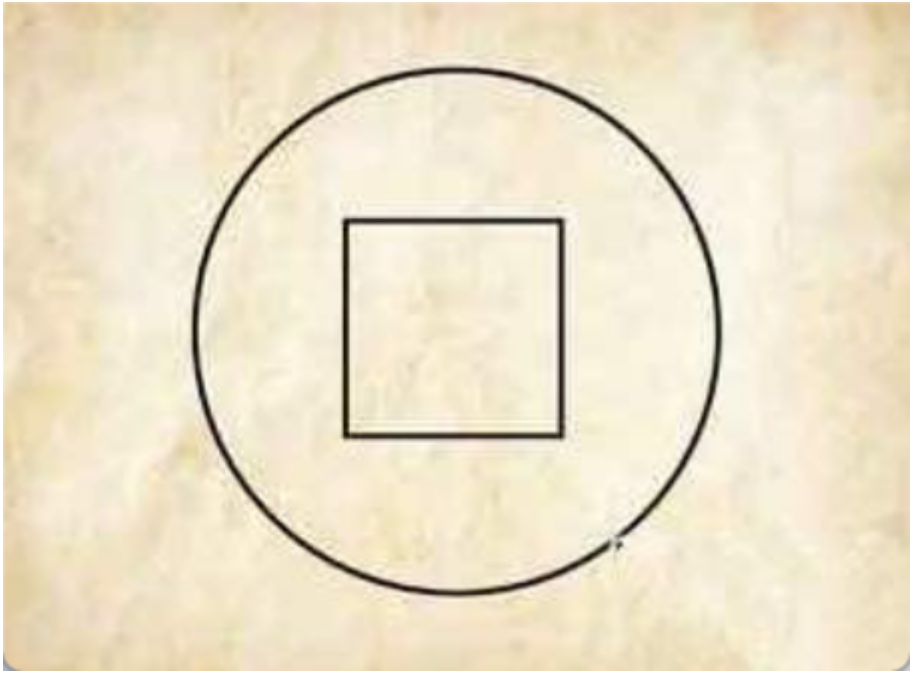
```
min_radius = round(min(image_height, image_width) / 10); % 示例：最小半径为短边的
1/10
max_radius = round(min(image_height, image_width) / 2);  % 示例：最大半径为短边的 1/2

sensitivity_factor = 0.90; % 灵敏度 (可调, 0.85-0.95 常用)
edge_thresh_factor = 0.1;  % 边缘阈值 (可调, 较低值可检测弱边缘圆)
obj_polarity = 'dark';    % 根据图像调整: 'bright' 或 'dark' (示例图中圆是黑线，背景亮)
% --- 参数调整结束 ---

[centers, radii, metric] = imfindcircles(gray_circle, [min_radius max_radius],
...
                                         'ObjectPolarity', obj_polarity, ...
                                         'Sensitivity', sensitivity_factor, ...
                                         'EdgeThreshold', edge_thresh_factor);

% 3. 显示检测结果
if ~isempty(centers)
    disp(['检测到 ', num2str(length(radii)), ' 个圆:']);
    for i = 1:length(radii)
        fprintf('  圆心: (%.1f, %.1f), 半径: %.1f, 度量值: %.2f\n', ...
                centers(i,1), centers(i,2), radii(i), metric(i));
    end

    hold on;
    % viscircles 是专门用来画圆的函数
    viscircles(centers, radii, 'EdgeColor', 'g', 'LineWidth', 2); % 用绿色绘制圆
    % 绘制圆心
    plot(centers(:,1), centers(:,2), 'r+', 'MarkerSize', 6, 'LineWidth', 2);
    hold off;
    title('改进后的霍夫圆检测结果');
else
    disp('没有检测到圆。');
    title('未检测到圆');
end
```



改进后的霍夫圆检测结果

